Projecte Bases de Dades

Bases de dades

Grup F

3 de juny del 2025

Jan Accensi March
David Armillas Gullón
Alex Cervantes Cabós
Cristian Prado Méndez

Índex

1 Introducció **2**

2 Objectius **2**

3 Model E-R **5**

4 Característiques de les dades **10**
4.1 Origen i recollida de les dades 10
4.2 Estructura de les dades 10
4.3 Característiques importants 10

5 Consultes amb AR **12**

6 Implementació **14**
6.1 Dades a Workbench 14
6.2 Queries 21
6.2.1 Gran premi amb final més just entre primer i segon classificat 21
6.2.2 Puntuació dels diferents equips per cada any 23
6.2.3 Puntuació dels diferents pilots per cada any 25

7 Anàlisi de les dades **28**

8 Conclusions i contribució dels membres **33**

A Codi Python pel tractament de dades **34**
A.1 Codi per passar CSV a Workbench 34
A.2 Codis per l'anàlisi 37

B Codi per crear la base de dades a Workbench **41**

1 Introducció

La Fórmula 1 representa la perfecta combinació entre el talent humà i l'excel·lència tecnològica. Des de la seva inauguració el 1950 a Silverstone, aquest campionat ha transcendit el simple concepte d'esport per convertir-se en un aparador global d'innovació en enginyeria. Al llarg de més de set dècades, les normatives tècniques han evolucionat contínuament, transformant no només els monoplaces sinó també la naturalesa mateixa de la competitivitat.

Històricament, les grans èpoques de l'F1 s'han definit tant pels canvis reglamentaris com pels pilots i equips que han dominat cada era. Des de l'època dels motors atmosfèrics fins a l'actual era híbrida, passant per revolucions aerodinàmiques com l'eliminació de les faldilles, les restriccions en alerons i difusors, o els recents canvis en la filosofia de flux d'aire amb l'efecte terra modern de 2022, cada període ha presentat un equilibri diferent entre la influència del pilot i la del vehicle. Aquestes transformacions constants no només en motorització sinó també en aerodinàmica i altres aspectes tècnics plantegen un repte fonamental: com podem discernir quines temporades han estat realment més competitives des d'una perspectiva purament d'enginyeria?

Convencionalment, la competitivitat s'ha mesurat mitjançant la distribució de punts entre constructors. Tanmateix, aquesta metodologia presenta una limitació crítica: l'efecte distorsionador del talent excepcional d'alguns pilots. Quan observem casos com el de Max Verstappen, que ha aconseguit campionats amb un Red Bull que no sempre ha estat clarament superior, o històricament Fernando Alonso lluint amb Ferrari en temporades on el cotxe vermell no era el més ràpid, sorgeix la pregunta: estem mesurant l'excel·lència en enginyeria o simplement el talent extraordinari d'un pilot capaç de superar les limitacions del seu monoplaça?

Per abordar aquest desafiament analític, utilitzarem una base de dades que inclou resultats de curses, puntuacions finals, dades sobre voltes ràpides, temps de pit-stop, i múltiples mètriques de rendiment tant per pilots com per constructors.

La nostra proposta metodològica es fonamenta en l'anàlisi estadística per distingir entre el rendiment intrínsec dels equips d'enginyeria i la contribució individual dels pilots. Mitjançant l'estudi de les diferències de rendiment entre companys d'equip —que teòricament disposen del mateix material— i l'aplicació de factors correctors, aspirem a obtenir una mesura ajustada que reflecteixi amb major precisió la competitivitat real entre els departaments tècnics d'enginyeria dels equips.

2 Objectius

Aquest projecte es proposa explorar i redefinir la nostra comprensió de la competitivitat a la Fórmula 1 des d'una perspectiva centrada en l'enginyeria. Pretenem desenvolupar una metodologia que permeti avaluar amb més precisió quines temporades i quines eres reglamentàries han estat realment més equilibrades des del punt de vista tècnic, eliminant la distorsió que generen els pilots excepcionals.

El nostre punt de partida serà l'anàlisi de la desviació estàndard dels punts acumulats pels constructors en cada temporada, establint així una mesura inicial de dispersió que ens indiqui com de proper ha estat el rendiment entre els diferents equips. No obstant això, reconeixem que aquesta mesura està influenciada pel factor humà, per la qual cosa el següent pas serà analitzar i quantificar sistemàticament les diferències de rendiment entre els pilots del mateix equip.

Aquesta anàlisi ens permetrà identificar aquells casos on un pilot està clarament sobrerendint respecte

al seu company d'equip, i per tant, potencialment emmascarant les deficiències del monoplaça. A partir d'aquesta informació, desenvoluparem un model matemàtic que aplicarà un factor corrector a la puntuació dels constructors, penalitzant aquells equips on la disparitat entre pilots sigui més gran. El resultat serà una puntuació ajustada que reflecteixi millor el rendiment del departament d'enginyeria.

Amb aquestes dades ajustades, recalcularem la competitivitat de cada temporada i, el que és més revelador, agruparem aquestes temporades segons els períodes normatius clau de l'F1 des de 1994:

- **L'Era de la Seguretat i l'Evolució Analògica (1994-2008)**: Període iniciat després de la tràgica mort de Senna, caracteritzat per millores progressives en seguretat i la reducció gradual d'ajudes electròniques.
- **L'Era de l'Eficiència Aerodinàmica (2009-2013)**: Marcada per la revolució aerodinàmica del 2009, la prohibició dels repostatges el 2010 i la introducció del DRS el 2011.
- **L'Era Híbrida Inicial (2014-2016)**: Definida pels motors V6 turbo híbrids que van transformar radicalment l'enginyeria de l'F1, prioritzant l'eficiència energètica.
- **L'Era de la Màxima Càrrega Aerodinàmica (2017-2021)**: Caracteritzada per cotxes més amples i ràpids, la implementació de l'Halo i l'inici del sostre pressupostari.
- **L'Era del Nou Efecte Terra (des de 2022)**: L'actual període amb monoplaces dissenyats per reduir la turbulència i facilitar els avançaments, combinat amb restriccions econòmiques més estrictes.

Aquesta estructuració ens permetrà comparar amb precisió quins conjunts de normatives han fomentat una major igualtat competitiva. Per exemple, es podrà veure si l'època post-Senna més competitiva que l'actual era de l'efecte terra o la diferència entre els diferents motors que s'han pogut veure en diferents normatives.

A més, aquesta metodologia ens oferirà una nova perspectiva sobre casos específics de dominació aparent, cosa que ens permet discernir entre aquells períodes on un equip va aconseguir una veritable superioritat tècnica (com podria ser el cas de Mercedes entre 2014 i 2016) i aquells on el factor determinant va ser principalment el pilot (com possiblement en algunes temporades de Red Bull amb Verstappen).

Per altra banda, hem de considerar que en aquests anys, la puntuació atorgada a les carreres ha anat canviant

En desenvolupar la nostra metodologia, hem d'abordar una complicació estadística significativa: els diversos sistemes de puntuació implementats per la FIA des de 1994:

- **1994-2002**: Puntuaven només els 6 primers classificats (10-6-4-3-2-1).
- **2003-2009**: Puntuaven els 8 primers classificats (10-8-6-5-4-3-2-1).
- **2010-actualitat**: Puntuen els 10 primers classificats (25-18-15-12-10-8-6-4-2-1).
- **2019-2024**: S'afegeix un punt addicional per volta ràpida al pilot que acabi entre els 10 primers.

Aquestes variacions fan que la desviació estàndard absoluta dels punts no sigui directament comparable entre temporades amb diferents escales de puntuació. Per exemple, la desviació natural serà més gran en l'era actual on el guanyador obté 25 punts que en l'era anterior on només n'obtenia 10.

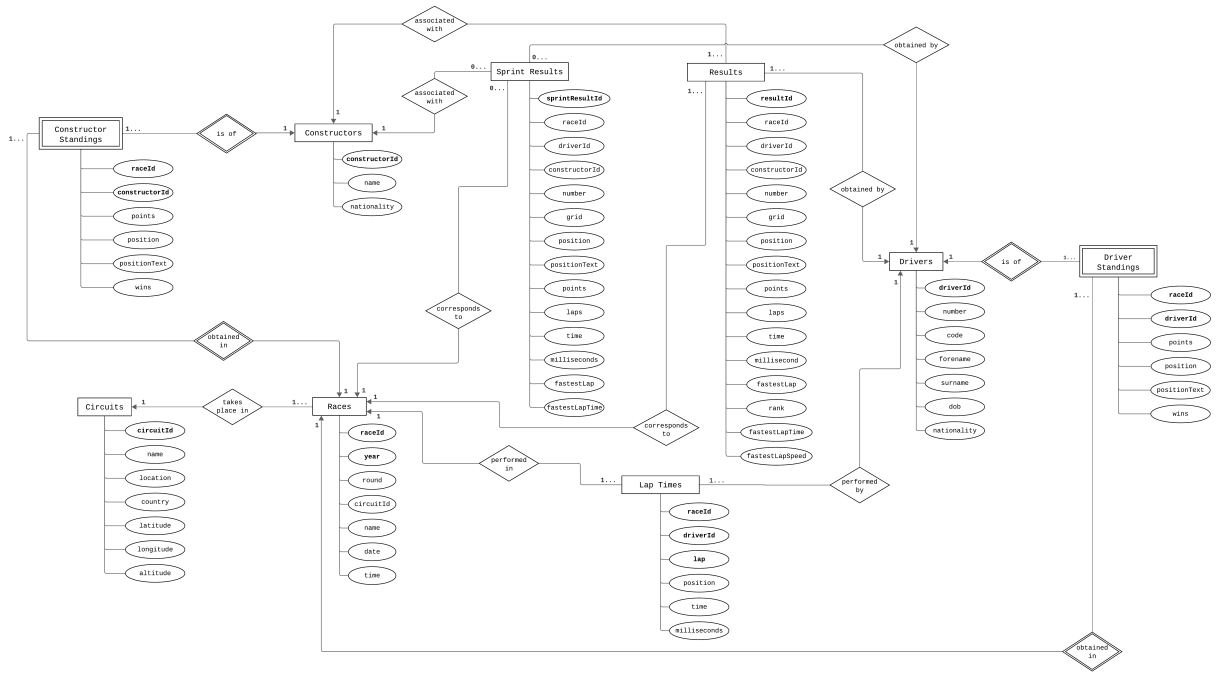
Per solucionar aquesta problemàtica, proposem utilitzar el **coeficient de variació (CV)** com a mesura primària de dispersió en lloc de la desviació estàndard absoluta, aconseguint que es puguin comparar diferents sistemes de puntuació amb diferents escales.

A partir d'aquest tractament de dades mitjançant les queries sobre la base de dades i la creació d'un programa per fer càlculs sobre aquestes dades, ens proposem respondre diverses preguntes com:

- Quina de les normatives va ser la que va provocar una competició més ajustada?
- Quin va ser l'any més competitiu en termes d'equips d'enginyeria?
- Es correlacionen els guanyadors reals de cada any amb els guanyadors merescuts (guanyadors un cop s'aplica la penalització)?
- Els mundials de Fernando Alonso (2005-2006) van ser més mèrit del cotxe/equip (Renault) o del seu talent?

3 Model E-R

Hem depurat les dades obtingudes per aconseguir el següent model, on apareixen totes les dades que ens haurien de fer falta per donar resposta al dubte plantejat als objectius:



Primer fem un repás de totes les entitats i els seus atributs:

- **Constructors**: Representa els equips que participen en les curses de Fórmula 1. Cada constructor està identificat per un identificador únic i inclou el nom i la nacionalitat de l'equip.
  - *constructorId*: Identificador únic del constructor.
  - *name*: Nom oficial de l'equip.
  - *nationality*: Nacionalitat de l'equip.
- **Constructor Standings [Dèbil]**: Emmagatzema la classificació dels constructors per a cada cursa. Indica els punts, la posició i les victòries obtingudes.
  - *raceId*: Identificador de la cursa corresponent.
  - *constructorId*: Referència al constructor.
  - *points*: Punts obtinguts pel constructor en la cursa.
  - *position*: Posició numèrica en la classificació.
  - *positionText*: Representació textual de la posició (pot incloure DNF, DSQ, etc.).
  - *wins*: Nombre de victòries assolides.
- **Drivers**: Inclou la informació personal dels pilots que competeixen en les curses. Cada pilot té un identificador únic, el seu nom complet, nacionalitat i altres dades rellevants.
  - *driverId*: Identificador únic del pilot.

  - *number*: Número de dorsal del pilot.
  - *code*: Codi abreujat del pilot.
  - *forename*: Nom del pilot.
  - *surname*: Cognom del pilot.
  - *dob*: Data de naixement.
  - *nationality*: Nacionalitat del pilot.

- **Driver Standings [Dèbil]**: Guarda la classificació individual dels pilots en cada cursa, indicant els punts, la posició i el nombre de victòries.
  - *raceId*: Referència a la cursa.
  - *driverId*: Referència al pilot.
  - *points*: Punts obtinguts.
  - *position*: Posició numèrica assolida.
  - *positionText*: Text representatiu de la posició.
  - *wins*: Nombre total de victòries acumulades.

- **Races**: Representa cada gran premi de Fórmula 1. Conté dades sobre la cursa com l'any (que és el que ho relacionarà amb cada temporada), la ronda, el circuit on es disputa i la seva data i hora.
  - *raceId*: Identificador únic de la cursa.
  - *year*: Any en què es disputa la cursa.
  - *round*: Número de la ronda dins la temporada.
  - *circuitId*: Referència al circuit on es fa la cursa.
  - *name*: Nom oficial de la cursa.
  - *date*: Data de la cursa.
  - *time*: Hora d'inici.

- **Circuits**: Emmagatzema la informació geogràfica dels circuits. S'inclouen el nom, ubicació, país i coordenades GPS.
  - *circuitId*: Identificador únic del circuit.
  - *name*: Nom oficial del circuit.
  - *location*: Localització o ciutat.
  - *country*: País del circuit.
  - *latitude*: Coordenada geogràfica (latitud).
  - *longitude*: Coordenada geogràfica (longitud).
  - *altitude*: Altitud sobre el nivell del mar.

- **Results**: Registra els resultats detallats de cada pilot en una cursa principal, incloent posició final, punts, temps, i dades sobre la volta més ràpida.

  - *resultId*: Identificador del resultat.
  - *raceId*: Referència a la cursa.
  - *driverId*: Referència al pilot.
  - *constructorId*: Referència al constructor.
  - *number*: Número del cotxe.
  - *grid*: Posició de sortida.
  - *position*: Posició final.
  - *positionText*: Text representatiu de la posició.
  - *points*: Punts guanyats.
  - *laps*: Voltes completades.
  - *time*: Temps final de cursa.
  - *millisecond*: Temps expressat en mil·lisegons.
  - *fastestLap*: Volta més ràpida realitzada.
  - *rank*: Rànquing de la volta més ràpida.
  - *fastestLapTime*: Temps de la volta més ràpida.
  - *fastestLapSpeed*: Velocitat de la volta més ràpida.

- **Lap Times**: Conté els temps de pas volta a volta de cada pilot per cada cursa. És útil per analitzar rendiment detallat.
  - *raceId*: Referència a la cursa.
  - *driverId*: Referència al pilot.
  - *lap*: Número de volta.
  - *position*: Posició en aquella volta.
  - *time*: Temps de pas (format text).
  - *milliseconds*: Temps en mil·lisegons.

- **Sprint Results**: Emmagatzema els resultats de les curses esprint, un format més curt de carrera introduït recentment. Aquesta carrera acostuma a fer-se el dia abans del gran premi.
  - *sprintResultId*: Identificador del resultat de sprint.
  - *raceId*: Referència a la cursa.
  - *driverId*: Referència al pilot.
  - *constructorId*: Referència al constructor.
  - *number*: Número del cotxe.
  - *grid*: Posició de sortida.
  - *position*: Posició final en la cursa sprint.
  - *positionText*: Posició en format textual.

- *points*: Punts obtinguts.
- *laps*: Voltes completades.
- *time*: Temps de finalització.
- *milliseconds*: Temps en mil·lisegons.
- *fastestLap*: Volta més ràpida assolida.
- *fastestLapTime*: Temps d'aquesta volta.

Hem marcat dues entitats com a dèbils: Constructor/Driver Standings, ja que només serveixen per relacionar un constructor o pilot amb una cursa i afegir-hi certes dades a la relació (el com han quedat). Fixada una parella constructor-cursa només hi ha un Constructor Standing que es relacioni amb aquests dos, per tant, una 'instància' de Constructor Standing queda definida unívocament pels identificadors del constructor i de la carrera. El mateix passa amb Driver Standing pels identificadors del conductor i de la carrera, per tant, aquestes dues entitats són dèbils.

Relació	**Cardinalitat**	**Justificació**
ConstructorStanding - Constructor	N:1	Cada entrada correspon a un únic constructor, però un constructor pot aparèixer en múltiples standings.
ConstructorStanding - Race	N:1	Cada standing és per una cursa concreta, i cada cursa té múltiples constructors amb puntuació.
DriverStanding - Driver	N:1	Cada resultat de standings és d'un sol pilot, però cada pilot pot aparèixer en múltiples curses.
DriverStanding - Race	N:1	Cada standing de pilot correspon a una cursa concreta, amb múltiples pilots participants.
Race - Circuit	N:1	Cada cursa té lloc en un únic circuit, però un circuit pot tenir múltiples curses.
Results - Race	N:1	Cada resultat és per una cursa concreta, però cada cursa té múltiples resultats.
Results - Driver	N:1	Cada resultat pertany a un pilot, però cada pilot pot tenir molts resultats.
Results - Constructor	N:1	Cada resultat està vinculat a un únic constructor, però un constructor pot tenir molts resultats.

SprintResult - Race	0...N:1	Cada resultat està vinculat a una carrera, i cada cursa té múltiples resultats de sprint, però les Sprints es van implementar fa pocs anys, llavors n'hi ha carreres que no tenen Sprints
SprintResult - Driver	0...N:1	Cada resultat d'esprint pertany a un pilot concret, i un pilot pot fer diversos resultats d'esprint, tot i que no tots els pilots han pilotat a l'època quan van sorgir les Sprints.
SprintResult - Constructor	0...N:1	Cada resultat d'esprint està associat a un únic constructor, i cada constructor pot tenir diversos resultats d'esprint, però hi han constructors que no van participar a les Sprints.
LapTime - Race	N:1	Cada volta es realitza en una cursa específica, però una cursa inclou moltes voltes.
LapTime - Driver	N:1	Cada temps de volta és d'un pilot, però un pilot pot tenir múltiples voltes.

4 Característiques de les dades

4.1 Origen i recollida de les dades

La base de dades Ergast conté informació completa sobre curses de Fórmula 1 des de l'any 1950 fins a l'any 2024. La base de dades està organitzada en diverses taules relacionades entre si. Aquestes dades, que ara tenim en format CSV, ofereixen un registre detallat de circuits, pilots, constructors, resultats i estadístiques de les competicions.

La base de dades va estar actualitzada setmanalment per un grup de voluntaris fins al 2024. Les dades es recollien de la pàgina web oficial formula1.com. Les dades es poden descarregar directament en un fitxer CSV. Si ens interessés obtenir ara com ara, també ofereixen una API jolpica-f1 amb la documentació necessària per a fer les crides.

Addicionalment, també volem incloure els anys que hi ha hagut un canvi de normativa en la competició. La font d'aquesta dada és pàgina Wikipedia Fórmula 1.

4.2 Estructura de les dades

Les dades s'organitzen en 9 taules principals. Cada fitxer CSV conté una única taula de base de dades. La codificació del text és UTF-8 i la primera línia de cada fitxer conté les capçaleres de les columnes:

1. **circuits**: Informació sobre els circuits (nom, ubicació, país, coordenades)
2. **constructors**: Equips d'F1 (nom, nacionalitat)
3. **drivers**: Pilots (nom, cognom, nacionalitat, data de naixement)
4. **races**: Curses (any, ronda, circuit, data, hora)
5. **results**: Resultats de les curses (posició, punts, temps, voltes)
6. **sprint_results**: Resultats de les curses de sprint (posició, punts, volta més ràpida, temps de la volta més ràpida)
7. **lap_times**: Temps per volta
8. **driver_standings**: Classificació de pilots
9. **constructor_standings**: Classificació d'equips

4.3 Característiques importants

- **Format temporal**: Dates, hores i durades en format ISO 8601, amb timezone UTC.
- **Codificació**: Les cadenes de text utilitzen codificació UTF-8.
- **Identificadors**: Els camps que acaben amb "Ref" són identificadors únics per a ús extern.
- **Estats especials**: S'utilitzen codis específics en el camp "positionText":
  - "D" - desqualificat

  - “E” - exclòs
  - “F” - no va classificar
  - “N” - no classificat
  - “R” - retirat
  - “W” - es va retirar

- **Graella de sortida**: Una posició de graella “0” indica sortida des del pit lane.

Aquestes dades CSV permeten dur a terme anàlisis detallades sobre rendiment de pilots i equips, estadístiques històriques, i comparatives entre diferents temporades de la Fórmula 1. Tot i la completesa de les dades, només utilitzarem aquelles entrades necessàries per a respondre a les preguntes dels objectius.

5 Consultes amb AR

En aquesta secció presentarem, a partir de les taules que utilitzarem en aquest projecte, algunes consultes que ens serviran per poder continuar amb el nostre estudi i d'altres que simplement serviran per trobar dades interessants. Considerem les taules esmentades a la secció 4.2.

- Imaginem un cas específic on hom vol saber en quin gran premi hi ha hagut el final més just entre el primer i el segon classificat. Considerem únicament les files dels primers i segons classificats:

$$\text{R_TOP2} \leftarrow \sigma_{\text{position}=1 \vee \text{position}=2}(\text{results})$$

  Ara, realitzem un equi-join entre els pilots que han quedat en primer i segon lloc a la mateixa cursa. Primer, es renomene (la rho) les dues instàncies de la taula R_TOP2 com R1 i R2 per poder diferenciar els primers i segons llocs. Després, es fa un join per emparellar els resultats de la mateixa carrera (R1.raceId = R2.raceId) i assegurar que un pilot és al primer lloc (R1.position = 1) i l'altre al segon (R2.position = 2). El resultat es guarda a la nova relació FINAL_JUST:

$$\text{FINAL_JUST} \leftarrow \rho(R1, \text{R_TOP2}) \bowtie_{\substack{\text{R1.raceId}=\text{R2.raceId} \\ \text{R1.position}=1 \\ \text{R2.position}=2}} \rho(R2, \text{R_TOP2})$$

  Generem la taula amb els id de les carreres i les diferències de temps i ho guardem a DIF_TEMPS:

$$\text{DIF_TEMPS} \leftarrow \pi_{\text{R1.raceId},(\text{R2.milliseconds}-\text{R1.milliseconds})\rightarrow\text{dif_temps}}(\text{FINAL_JUST})$$

  Ordenem els resultats per dif_temps i agafem el més petit:

$$\text{MIN_DIF} \leftarrow \sigma_{\text{dif_temps}=\text{mín}(\pi_{\text{dif_temps}}(\text{DIF_TEMPS}))}(\text{DIF_TEMPS})$$

  Finalment, fem un join per obtenir el nom del gran premi:

$$\text{RESULTAT} \leftarrow \text{MIN_DIF} \bowtie_{\text{raceId}} \pi_{\text{raceId,name}}(\text{races})$$

- Considerem ara una consulta que sí que necessitarem en un futur. La consulta està relacionada amb els punts acumulats per constructors des de l'any 1994. Filtrarem la taula races per quedar-nos només amb les curses disputades des de 1994:

$$\text{GP_1994} \leftarrow \sigma_{\text{year}\geq 1994}(\text{races})$$

  Fem un join entre races_1994 i results per obtenir els resultats de cada carrera:

$$\text{RESULTATS_1994} \leftarrow \text{results} \bowtie_{\text{raceId}} \text{GP_1994}$$

  Afegim la informació dels constructors per cada resultat del gran premi:

$$\text{RES_CONST} \leftarrow \text{RESULTATS_1994} \bowtie_{\text{constructorId}} \text{constructors}$$

Sumem els punts de cada constructor per cada any, utilitzem l'operació γ, que ens serveix per agrupar per constructorId i year amb els punts corresponents (s'utilitza perquè d'altra manera seria molt farragós):

$$\text{PUNTS_CONST} \leftarrow \gamma_{\text{constructorId, year,}\sum\text{(points)}\rightarrow\text{total_points}}(\text{RES_CONST})$$

Finalment, amb la taula de PUNTS_CONST que conté l'any, l'id del constructor i els punts, afegim els noms dels constructors amb un join:

$$\text{PUNTS_FINAL} \leftarrow \text{PUNTS_CONST} \bowtie_{\text{constructorId}} \pi_{\text{constructorId,name}}(\text{constructors})$$

- Sigui ara el cas que volem saber la mateixa informació, però ara dels pilots. Les consultes seran bastant similars:

  Seleccionem les curses que van tenir lloc el 1994 o després:

  $$\text{GP_1994} \leftarrow \sigma_{\text{year}\geq 1994}(\text{races})$$

  Fem un join i obtenim els resultats des del 1994:

  $$\text{RESULTATS_1994} \leftarrow \text{results} \bowtie_{\text{raceId}} \text{GP_1994}$$

  Obtenim els resultats dels pilots:

  $$\text{RES_PILOTS} \leftarrow \text{RESULTATS_1994} \bowtie_{\text{driverId}} \text{drivers}$$

  Els punts a partir dels resultats:

  $$\text{PUNTS_PILOTS} \leftarrow \gamma_{\text{driverId, year,}\sum\text{(points)}\rightarrow\text{total_points}}(\text{RES_PILOTS})$$

  I finalment, obtenim la taula amb el pilot i els seus resultats al llarg de les temporades:

  $$\text{PUNTS_FINAL} \leftarrow \text{PUNTS_PILOTS} \bowtie_{\text{driverId}} \pi_{\text{driverId,forename,surname}}(\text{drivers})$$

6 Implementació

6.1 Dades a Workbench

A la mateixa pàgina on vam extreure les dades ja existia un arxiu sql que ens servia per poder crear les taules al workbench, tot i això, vam decidir crear un des de zero per poder definir les taules i atributs que ja vam definir al mateix model entitat-relació. Anirem pas a pas analitzant el codi desenvolupat. Afegirem el codi sencer al final del document, a un annex.
En primer lloc, el que fem és guardar el valor original del sistema a una variable temporal `SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS` i posteriorment desactivant la verificació d'unicitat `UNIQUE_CHECKS=0` (amb això podem afegir dades que potencialment poden trencar la restricció de clau única). Tot seguit, fem el mateix amb les claus foranes.

```
SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
```

Establim un seguit de modes SQL, permeten controlar el comportament davant alguns error (no permetre dividir entre 0, no tenir 00 a les dates...).

```
SET @OLD_SQL_MODE=@@SQL_MODE, ...
    SQL_MODE='STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE, ...
    ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';
```

Generem la base de dades amb nom `f1db`, si ja existeix s'elimina. A més, definim que s'utilitzi el sistema Unicode i a partir de `utf8mb4_unicode_ci` definim unes regles de comparació entre texts.

```
DROP SCHEMA IF EXISTS `f1db`;
CREATE SCHEMA IF NOT EXISTS `f1db` DEFAULT CHARACTER SET utf8mb4 COLLATE ...
    utf8mb4_unicode_ci;
USE f1db;
```

Comencem a afegir taules a la nova base de dades. Si no existeix, s'afegeix la taula `circuits`, la qual té els atributs i claus primàries definides al mateix model E-R. Fixem-nos que hi han camps que NO poden ser null, ja que no té molt sentit que un circuit no tingui nom o que un pilot no tingui cognom.

```
-- CIRCUITS
CREATE TABLE IF NOT EXISTS `f1db`.`circuits` (
  `circuitId` INT NOT NULL,
  `name` VARCHAR(255) NOT NULL,
  `location` VARCHAR(255),
  `country` VARCHAR(255),
  `latitude` FLOAT,
  `longitude` FLOAT,
  `altitude` INT,
  CONSTRAINT `circuit_pkey` PRIMARY KEY (`circuitId`)
```

```
);
```

Mantenim el mateix raonament amb les següents taules.

```
-- CONSTRUCTORS
CREATE TABLE IF NOT EXISTS `f1db`.`constructors` (
  `constructorId` INT NOT NULL,
  `name` VARCHAR(255) NOT NULL,
  `nationality` VARCHAR(255),
  CONSTRAINT `constructor_pkey` PRIMARY KEY (`constructorId`)
);
```

Mecionar que, per exemple, el `code` (sigles de tres lletres que serveixen per identificar a un pilot sense utilitzar el nom sencer, ex: Alonso - ALO, Carlos Sainz - SAI) d'un pilot pot no existir, ja que no sempre s'han utilitzat.

```
-- DRIVERS
CREATE TABLE IF NOT EXISTS `f1db`.`drivers` (
  `driverId` INT NOT NULL,
  `number` INT,
  `code` VARCHAR(3),
  `forename` VARCHAR(255) NOT NULL,
  `surname` VARCHAR(255) NOT NULL,
  `dob` DATE,
  `nationality` VARCHAR(255),
  CONSTRAINT `driver_pkey` PRIMARY KEY (`driverId`)
);
```

```
-- RACES
CREATE TABLE IF NOT EXISTS `f1db`.`races` (
  `raceId` INT NOT NULL,
  `year` INT NOT NULL,
  `round` INT NOT NULL,
  `circuitId` INT NOT NULL,
  `name` VARCHAR(255) NOT NULL,
  `date` DATE,
  `time` TIME,
  CONSTRAINT `race_pkey` PRIMARY KEY (`raceId`)
);
```

Aquestes dues taules són pràcticament similars, només canvien un parell de coses.

```
-- DRIVER STANDINGS
CREATE TABLE IF NOT EXISTS `f1db`.`driverStandings` (
  `raceId` INT NOT NULL,
  `driverId` INT NOT NULL,
```

```
  `points` FLOAT NOT NULL DEFAULT 0,
  `position` INT,
  `positionText` VARCHAR(255),
  `wins` INT DEFAULT 0
);

-- CONSTRUCTOR STANDINGS
CREATE TABLE IF NOT EXISTS `f1db`.`constructorStandings` (
  `raceId` INT NOT NULL,
  `constructorId` INT NOT NULL,
  `points` FLOAT NOT NULL DEFAULT 0,
  `position` INT,
  `positionText` VARCHAR(255),
  `wins` INT DEFAULT 0
);
```

Bastant interessant trobar a la següent taula que `laps` pugui ser null, però té una explicació, ja que a vegades un gran premi es pot suspendre o fins i tot que no hagi temps suficient per fer totes les voltes (hi ha un temps límit d'ocupació de la pista, si pel que sigui es comença massa tard i no existeix temps físic per fer totes les voltes, aleshores el gran premi anirà en funció d'un temps determinat).

```
-- RESULTS
CREATE TABLE IF NOT EXISTS `f1db`.`results` (
  `resultId` INT NOT NULL,
  `raceId` INT NOT NULL,
  `driverId` INT NOT NULL,
  `constructorId` INT NOT NULL,
  `number` INT,
  `grid` INT,
  `position` INT,
  `positionText` VARCHAR(255),
  `points` FLOAT,
  `laps` INT,
  `time` VARCHAR(255),
  `milliseconds` INT,
  `fastestLap` INT,
  `rank` INT,
  `fastestLapTime` VARCHAR(255),
  `fastestLapSpeed` VARCHAR(255),
  CONSTRAINT `results_pkey` PRIMARY KEY (`resultId`)
);

-- SPRINT RESULTS
CREATE TABLE IF NOT EXISTS `f1db`.`sprintResults` (
  `sprintResultId` INT NOT NULL,
  `raceId` INT NOT NULL,
  `driverId` INT NOT NULL,
  `constructorId` INT NOT NULL,
```

```
28          `number` INT,
29          `grid` INT,
30          `position` INT,
31          `positionText` VARCHAR(255),
32          `points` FLOAT,
33          `laps` INT,
34          `time` VARCHAR(255),
35          `milliseconds` INT,
36          `fastestLap` INT,
37          `fastestLapTime` VARCHAR(255),
38          CONSTRAINT `sprintresults_pkey` PRIMARY KEY (`sprintResultId`)
39        );
```

```
1         -- LAP TIMES
2         CREATE TABLE IF NOT EXISTS `f1db`.`lapTimes` (
3           `raceId` INT NOT NULL,
4           `driverId` INT NOT NULL,
5           `lap` INT NOT NULL,
6           `position` INT,
7           `time` VARCHAR(255),
8           `milliseconds` INT,
9           CONSTRAINT `laptimes_pkey` PRIMARY KEY (`raceId`, `driverId`, `lap`)
10        );
```

A les següents línies definim les claus foranes. S'estableix el mode `ON DELETE CASCADE` que permet l'eliminació dels registres relacionats a les taules filles si s'elimina a la taula pare.

```
1         -- CLAVES FORNEAS
2
3         ALTER TABLE `f1db`.`races`
4           ADD CONSTRAINT `races_fk_circuit` FOREIGN KEY (`circuitId`) REFERENCES ...
              `f1db`.`circuits` (`circuitId`) ON DELETE CASCADE;
5
6         ALTER TABLE `f1db`.`driverStandings`
7           ADD CONSTRAINT `ds_fk_race` FOREIGN KEY (`raceId`) REFERENCES `f1db`.`races` ...
              (`raceId`) ON DELETE CASCADE,
8           ADD CONSTRAINT `ds_fk_driver` FOREIGN KEY (`driverId`) REFERENCES ...
              `f1db`.`drivers` (`driverId`) ON DELETE CASCADE;
9
10        ALTER TABLE `f1db`.`constructorStandings`
11          ADD CONSTRAINT `cs_fk_race` FOREIGN KEY (`raceId`) REFERENCES `f1db`.`races` ...
              (`raceId`) ON DELETE CASCADE,
12          ADD CONSTRAINT `cs_fk_constructor` FOREIGN KEY (`constructorId`) REFERENCES ...
              `f1db`.`constructors` (`constructorId`) ON DELETE CASCADE;
13
14        ALTER TABLE `f1db`.`results`
15          ADD CONSTRAINT `results_fk_race` FOREIGN KEY (`raceId`) REFERENCES ...
```

```
            `f1db`.`races` (`raceId`) ON DELETE CASCADE,
        ADD CONSTRAINT `results_fk_driver` FOREIGN KEY (`driverId`) REFERENCES ...
            `f1db`.`drivers` (`driverId`) ON DELETE CASCADE,
        ADD CONSTRAINT `results_fk_constructor` FOREIGN KEY (`constructorId`) REFERENCES ...
            `f1db`.`constructors` (`constructorId`) ON DELETE CASCADE;

      ALTER TABLE `f1db`.`sprintResults`
        ADD CONSTRAINT `sprint_fk_race` FOREIGN KEY (`raceId`) REFERENCES `f1db`.`races` ...
            (`raceId`) ON DELETE CASCADE,
        ADD CONSTRAINT `sprint_fk_driver` FOREIGN KEY (`driverId`) REFERENCES ...
            `f1db`.`drivers` (`driverId`) ON DELETE CASCADE,
        ADD CONSTRAINT `sprint_fk_constructor` FOREIGN KEY (`constructorId`) REFERENCES ...
            `f1db`.`constructors` (`constructorId`) ON DELETE CASCADE;

      ALTER TABLE `f1db`.`lapTimes`
        ADD CONSTRAINT `laptime_fk_race` FOREIGN KEY (`raceId`) REFERENCES ...
            `f1db`.`races` (`raceId`) ON DELETE CASCADE,
        ADD CONSTRAINT `laptime_fk_driver` FOREIGN KEY (`driverId`) REFERENCES ...
            `f1db`.`drivers` (`driverId`) ON DELETE CASCADE;
```

Restaurem les restriccions originals, es tornen a activar les restriccions d'unicitat i de claus foranes per garantir la integritat referencial i la unicitat de les dades.

```
      SET SQL_MODE=@OLD_SQL_MODE;
      SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
      SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
```

Un cop tenim la base de dades creada al Workbench, només necessitem passar les dades dels CSV a la base `f1db`. Per fer això, hem utilitzat el codi que podem trobar a l'annex A.1. De fet, el codi proporcionat ens permet generar la base de dades des de fora, és a dir, si executem el codi (afegint la contrasenya i el host) es crea la base de dades i s'afegeixen les dades directament. Al codi hem utilizar les llibreries `pandas` i `mysql.connector`, que permeten tractar els csv com a taules (i filtrar-les) i afegir i crear les taules/dades al `MySQLWorkbench`. A les primeres línies del codi generem la base de dades, això ho fem amb les línies

```
      schema_path = "f1db_tables_final.sql"

      # Configuraci n de conexi n
      conn = mysql.connector.connect(
          host="localhost",
          user="root",
          password="****", # Poner la contrase a de tu db
      )

      # Crear un cursor para ejecutar consultas
      cursor = conn.cursor()

      # Leer el archivo SQL y ejecutar las consultas
```

```
        with open(schema_path, "r", encoding="utf-8") as f:
            sql_script = f.read()

        # Ejecutar m ltiples sentencias separadas por ";"
        for statement in sql_script.split(";"):
            stmt = statement.strip()
            if stmt:
                try:
                    cursor.execute(stmt)
                except mysql.connector.Error as err:
                    print(f"Error ejecutando sentencia: {stmt}\n{err}")

        conn.commit()
        cursor.close()
        conn.close()
```

Que executen les línies que trobem a l'arxiu `f1db_tables_final.sql`. Un cop tenim això simplement definim les dades que volem de cada CSV

```
        tables = {
            "Constructors": ["constructorId", "name", "nationality"],
            "Drivers": ["driverId", "number", "code", "forename", "surname", "dob", ...
                "nationality"],
            "Circuits": ["circuitId", "name", "location", "country", "latitude", ...
                "longitude", "altitude"],
            "Races": ["raceId", "year", "round", "circuitId", "name", "date", "time"],
            "Results": ["resultId", "raceId", "driverId", "constructorId", "number", ...
                "grid", "position", "positionText", "points", "laps", "time", ...
                "milliseconds", "fastestLap", "fastestLapTime", "fastestLapSpeed"],
            "SprintResults": ["sprintResultId", "raceId", "driverId", "constructorId", ...
                "number", "grid", "position", "positionText", "points", "laps", "time", ...
                "milliseconds", "fastestLap", "fastestLapTime"],
            "LapTimes": ["raceId", "driverId", "lap", "position", "time", "milliseconds"],
            "ConstructorStandings": ["raceId", "constructorId", "points", "position", ...
                "positionText", "wins"],
            "DriverStandings": ["raceId", "driverId", "points", "position", ...
                "positionText", "wins"]
        }
```

I comencem a tractar cada CSV amb un `for`, afegint totes les dades que necessitem.

```
        # Procesar cada tabla
        for table, valid_cols in tables.items():
            # Obtener el nombre del archivo correspondiente
            data_folder = "data"
            file_name = os.path.join(data_folder, file_map.get(table))

            # Comprobar si el archivo existe
```

```
            if not os.path.exists(file_name):
                print(f"Archivo no encontrado para {table}: {file_name}")

            print(f"Insertando en tabla {table} desde {file_name}...")

            # Leer el archivo CSV
            # Se a ade el argumento na_values para manejar los valores nulos
            df = pd.read_csv(file_name, na_values='\\N')

            # Manejo especial para SprintResults: a adir sprintResultId , ya que de por ...
                s   no existe
            if table == "SprintResults" and "sprintResultId" not in df.columns:
                print("A adiendo sprintResultId autogenerado...")
                df.insert(0, "sprintResultId", range(1, len(df) + 1))

            # Filtrar columnas v lidas que existen en el archivo
            df = df[[col for col in valid_cols if col in df.columns]]

            # Eliminar duplicados por clave primaria (primera columna del schema)
            df = df.drop_duplicates(subset=df.columns[0], keep="first")

            # Reemplazar NaN con None (importante para MySQL)
            df = df.where(pd.notnull(df), None)

            # Construcci n din mica de columnas y placeholders
            insert_cols = df.columns.tolist()
            col_names = ', '.join([f"`{col}`" for col in insert_cols])
            placeholders = ', '.join(['%s'] * len(insert_cols))

            # Por cada fila
            for _, row in df.iterrows():

                # Crear una tupla con los valores de la fila
                row_tuple = tuple([None if pd.isna(val) else val for val in row])

                # Insertar en la tabla
                cursor.execute(
                    f"INSERT INTO {table} ({col_names}) VALUES ({placeholders})",
                    row_tuple
                )
            # Commit de la transacci n
            conn.commit()
            print(f"Insertados {len(df)} registros en {table}")

        # Cerrar la conexi n
        cursor.close()
        # Cerrar la conexi n
        conn.close()
```

Amb això fet, podem començar a realitzar les queries necessàries per poder fer l'estudi.

6.2 Queries

Les queries que s'han realitzat son una adaptació de les consultes amb àlgebra relacional que es van proposar a la primera part del projecte. Tal com es podrà veure, la segona i la tercera consulta seran essencials pel tractament de dades que es farà després en el seu anàlisi. A partir d'aquestes queries, s'obtindran les taules en format CSV, que contindran dades dels pilots i dels equips des de l'any 1994. Aquestes taules seran tractades en un arxiu Python, que ens permetrà respondre les preguntes que es van plantejar.

Notem que sempre farem ús de la línia de codi `DROP VIEW IF EXISTS nomTaula;` per facilitar la gestió en el cas que es volguessin fer modificacions a les queries per obtenir noves dades/atributs.

6.2.1 Gran premi amb final més just entre primer i segon classificat

Per tal de poder saber aquesta informació, en farem ús de 4 queries. En aquesta primera pregunta, volem saber quin ha estat el gran premi amb final més just entre primer i segon classificat de tota la història de la Formula 1.

En primer lloc, el que volem és filtrar la taula de `results`, on només volem obtenir els resultats que corresponen a la primera o segona posició de totes les carreres. Per conveniència, el que farem serà ordenar en ordre ascendent les dades mitjançant el `raceId`, a partir de `ORDER BY`.

```
DROP VIEW IF EXISTS R_TOP2;

CREATE VIEW R_TOP2 AS
SELECT
    *
FROM
    results
WHERE
    position = 1 OR position = 2
ORDER BY
    raceId ASC;
```

Seguidament, una vegada ja tenim els resultats de la primer i la segona posició de cada carrera, el que farem serà obtenir dades de les carreres i els temps que van trigar el primer i el segon classificat de cada carrera, en mil·lisegons. Per fer això, es farà ús de la View `R_TOP2` i es farà un Auto-Join, on filtrarem aquelles files on coincideixi el `raceId`. Per poder mantenir un ordre i evitar files duplicades respecte a les dades que es necessiten, el que es farà serà que a la taula de l'esquerra del `JOIN` hi hagi el `raceId` del primer classificat i a la de la dreta el del segon. Finalment, ordenem pel raceId en ordre ascendent amb `ORDER BY`.

```
DROP VIEW IF EXISTS FINAL_JUST;

CREATE VIEW FINAL_JUST AS
SELECT
    R1.raceId,
    R1.driverId AS driver1,
    R1.milliseconds AS temps1,
    R2.driverId AS driver2,
```

```
            R2.milliseconds AS temps2
        FROM
            R_TOP2 AS R1
        JOIN
            R_TOP2 AS R2
            ON R1.raceId = R2.raceId
        WHERE
            R1.position = 1
            AND R2.position = 2
        ORDER BY
            R1.raceId ASC;
```

En aquest moment, ja es té que per cada carrera tenim els temps del primer i del segon classificat. Per tant, ara es vol obtenir la carrera amb la mínima diferència de temps entre el primer i el segon classificat. Per això, es farà la resta de mil·lisegons entre el segon i el primer classificat. Notem que es fa la resta del temps del segon - temps del primer, ja que així evitem els nombres negatius. Això es farà a partir de la view anterior `FINAL_JUST`. Es podria donar el cas que hi haguessin diverses carreres amb la mateixa diferència de temps. Per tant el que es farà serà ordenar per ordre ascendent de `raceId`.

```
        DROP VIEW IF EXISTS DIF_TEMPS;

        CREATE VIEW DIF_TEMPS AS
        SELECT
            raceId,
            (temps2 - temps1) AS dif_temps
        FROM
            FINAL_JUST
        ORDER BY
            raceId ASC;
```

Arribats a aquest punt, cal comentar que hem estalviat una consulta respecta a l'àlgebra relacional. Aquí s'han ajuntat dues consultes. En particular, s'ha fet un `JOIN` entre la view `DIF_TEMPS` i la taula `races`. D'aquí es tindran aquelles files amb el mateix `raceId` de R1 i R2. Finalment, es seleccionaran aquelles files on la diferència de temps sigui igual a la mínima que es té a la taula `DIF_TEMPS`.

```
        DROP VIEW IF EXISTS MIN_DIF;

        CREATE VIEW MIN_DIF AS
        SELECT R2.name, R2.date
        FROM DIF_TEMPS AS R1
        INNER JOIN races AS R2 ON R1.raceId = R2.raceId
        WHERE R1.dif_temps = (
            SELECT MIN(dif_temps)
            FROM DIF_TEMPS
        );
```

Per finalitzar, veiem que obtenim el següent resultat fent la comanda `SELECT * FROM MIN_DIF;`

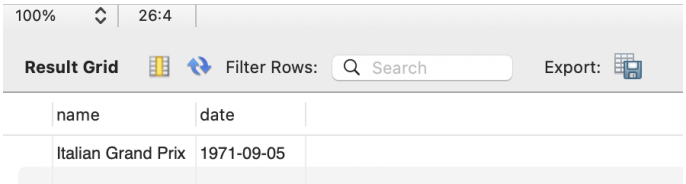


Figura 1: Carrera amb menor diferència entre primer i segon classificat de Formula 1

En efecte, es corrobora el resultat fent una cerca per internet, on consta que el gran premi més ajustat va ser aquest.

6.2.2 Puntuació dels diferents equips per cada any

Aquesta segona query és una de les queries més importants per poder fer l'anàlisi. En aquest cas es busca obtenir els punts acumulats per cada equip a cada any. En particular, cal comentar que NO s'inclouen els punts acumulats a les carreres Sprint. Per tant, la puntuació obtinguda pels equips serà únicament donada de les carreres estàndard.

Per tant, en primer lloc agafarem les carreres on centrem l'estudi. En particular, de la taula `races` agafem aquelles que són de l'any 1994 cap endavant. Ordenarem per l'any de la carrera en ordre ascendent, mitjançant `ORDER BY` i ho guardarem a la view `GP_1994`

```
DROP VIEW IF EXISTS GP_1994;

CREATE VIEW GP_1994 AS
SELECT
    *
FROM
    races
WHERE
    year >= 1994
ORDER BY
    year ASC;
```

Seguidament, farem ús de `INNER JOIN` per poder relacionar la view `GP_1994` amb la taula `results`. En particular, el que volem obtenir és una nova vista que anomenarem `RESULTATS_1994` on es guardaran totes les dades de les carreres i afegirem també el `driverId`, `constructorId` i els punts de cada pilot en aquella carrera, gràcies a la taula `results`. Clarament, filtrarem les files que compleixin que el raceId és el mateix. Un altre cop, ordenarem per any de carrera, en ordre ascendent.

```
DROP VIEW IF EXISTS RESULTATS_1994;

CREATE VIEW RESULTATS_1994 AS
SELECT
    R2.*,
    R1.driverId,
    R1.constructorId,
```

```
            R1.points
        FROM
            results AS R1
        INNER JOIN
            GP_1994 AS R2
            ON R1.raceId = R2.raceId
        ORDER BY
            year ASC;
```

Ara, utilitzarem la view `RESULTATS_1994` per poder fer un `INNER JOIN` amb `constructors`. D'aquesta forma, obtindrem una nova view anomenada `RES_CONST` on es tindran com atributs tots els de la view `RESULTATS_1994` i el nom i la nacionalitat de l'equip, a partir de la taula `constructors`. Per això, el que farem serà filtrar les files, on es tindran aquelles on coincideixi el `constructorId` de les dues taules. Ordenarem com sempre per any, en ordre ascendent.

```
        DROP VIEW IF EXISTS RES_CONST;

        CREATE VIEW RES_CONST AS
        SELECT
            R2.*,
            R1.name AS constructor_name,
            R1.nationality
        FROM
            constructors AS R1
        INNER JOIN
            RESULTATS_1994 AS R2
            ON R1.constructorId = R2.constructorId
        ORDER BY
            year ASC;
```

Una vegada es tenen tots els resultats de cada carrera amb els punts obtinguts per cada pilot i la seva escuderia, es vol obtenir la puntuació obtinguda per cada equip en aquella temporada. Per això es farà servir la view `RES_CONST`, creada anteriorment. En particular, es farà un `SELECT` del `constructorId`, el nom de l'escuderia, l'any i finalment un últim atribut anomenat `total_points`, que serà la suma de tots els punts obtinguts per l'equip en aquella temporada/any. Per això, es tenen en compte tots els pilots que han format part de l'escuderia en aquell any. Per poder fer això, es farà servir `GROUP BY`, que ens servirà per tractar la funció d'agregació `SUM`. En particular, agruparem pel constructor Id i per l'any. D'aquesta forma, obtindrem les dades ben ordenades i tindrem la suma de punts total de cada equip en una temporada concreta. Per conveniència, tornarem a ordenar per any de forma ascendent.

```
        DROP VIEW IF EXISTS PUNTS_CONST;

        CREATE VIEW PUNTS_CONST AS
        SELECT
            constructorId,
            constructor_name,
```

```
            year,
            SUM(points) AS total_points
        FROM
            RES_CONST
        GROUP BY
            constructorId,
            year
        ORDER BY
            year ASC;
```

I obtindrem la següent vista, d'on extraurem el primer arxiu CSV dels dos que necessitarem per fer l'anàlisi de dades. Fent ús de la comanda `SELECT * FROM PUNTS_CONST;`

100% 26:1

Result Grid Filter Rows: Search Export:

constructorId	constructor_name	year	total_points
22	Benetton	1994	103
3	Williams	1994	118
6	Ferrari	1994	71
17	Jordan	1994	28
25	Tyrrell	1994	13
15	Sauber	1994	12
32	Team Lotus	1994	0
18	Minardi	1994	5
33	Larrousse	1994	2
27	Ligier	1994	13
31	Simtek	1994	0
1	McLaren	1994	42
29	Footwork	1994	9
30	Pacific	1994	0
22	Benetton	1995	147
3	Williams	1995	118

Figura 2: Puntuacions per equip de cada temporada

6.2.3 Puntuació dels diferents pilots per cada any

En aquest cas es busca obtenir els punts acumulats per cada pilot a cada any. En particular, cal comentar que NO s'inclouen els punts acumulats a les carreres Sprint. Per tant, la puntuació obtinguda pels pilots serà únicament donada de les carreres estàndard.

Tal com es va advertir a la part de les consultes amb àlgebra relacional, trobar la puntuació dels diferents equips per cada any i les dels pilots és molt similar. En particular, les dues primeres queries (creació de vistes) és idèntica. Per tant, el codi que utilitzarem és el mateix

```
        DROP VIEW IF EXISTS GP_1994;

        CREATE VIEW GP_1994 AS
        SELECT
            *
        FROM
            races
        WHERE
```

```
            year >= 1994
        ORDER BY
            year ASC;

        DROP VIEW IF EXISTS RESULTATS_1994;

        CREATE VIEW RESULTATS_1994 AS
        SELECT
            R2.*,
            R1.driverId,
            R1.constructorId,
            R1.points
        FROM
            results AS R1
        INNER JOIN
            GP_1994 AS R2
            ON R1.raceId = R2.raceId
        ORDER BY
            year ASC;
```

Arribats a aquest punt, es voldrà obtenir el code amb el que s'identifica el pilot, nom, cognom i nacionalitat, així com les dades que ja es tenen a la vista `RESULTATS_1994`. Per això, es farà ús d'un `INNER JOIN` de la vista `RESULTATS_1994` amb la taula `drivers`. Clarament, es seleccionaran aquelles files on coincideixin el `driverId` d'ambdues. Finalment, s'ordenaran les dades per any, en ordre ascendent.

```
        DROP VIEW IF EXISTS RES_PILOTS;

        CREATE VIEW RES_PILOTS AS
        SELECT
            R2.*,
            R1.code,
            R1.forename,
            R1.surname,
            R1.nationality
        FROM
            drivers AS R1
        INNER JOIN
            RESULTATS_1994 AS R2
            ON R1.driverId = R2.driverId
        ORDER BY
            year ASC;
```

De forma similar a com es va fer en el cas dels equips, es farà ús de la vista creada `RES_PILOTS` i es farà una selecció de l'any, el driverId, el nom i cognom del pilot, la suma dels seus punts en aquella temporada i el nombre de carreres que va fer el pilot aquell any amb l'escuderia. Observem que es fan servir les funcions d'agregació `SUM` i `COUNT`. Per tant, es fa ús de `GROUP BY` per poder agrupar les dades a partir del constructorId, l'any i el driverId. Notem que es fa servir el constructorId perquè un pilot al llarg de la seva carrera ha estat en diversos equips. Però no només això, sinó que hi ha casos on el pilot ha estat en

diversos equips a la mateixa temporada. Per tant, necessitem agrupar a partir d'aquests 3 atributs. Com a comentari, cal dir que tant els punts com el nombre de carreres seran importants per fer l'anàlisi, ja que es farà una mitjana de punts per carrera de cada pilot a la temporada. D'aquesta forma, es podrà obtenir una penalització molt més justa per l'equip.

Finalment ordenem les dades a partir de l'any en ordre ascendent.

```sql
DROP VIEW IF EXISTS PUNTS_PILOTS;

CREATE VIEW PUNTS_PILOTS AS
SELECT
    constructorId,
    year,
    driverId,
    forename,
    surname,
    SUM(points) AS total_points,
    COUNT(driverId) AS num_races
FROM
    RES_PILOTS
GROUP BY
    constructorId,
    year,
    driverId
ORDER BY
    year ASC;
```

I obtindrem la següent vista, d'on extraurem el segon arxiu CSV que necessitarem per fer l'anàlisi de dades. Fent ús de la comanda `SELECT * FROM PUNTS_PILOTS;`

100% 27:1

Result Grid Filter Rows: Search Export:

constructorId	year	driverId	forename	surname	total_points	num_races
22	1994	30	Michael	Schumacher	92	14
3	1994	71	Damon	Hill	91	16
6	1994	55	Jean	Alesi	24	14
17	1994	22	Rubens	Barrichello	19	16
25	1994	79	Ukyo	Katayama	5	16
15	1994	91	Karl	Wendlinger	4	4
32	1994	65	Johnny	Herbert	0	13
18	1994	94	Pierluigi	Martini	4	16
33	1994	100	Érik	Comas	2	15
32	1994	83	Pedro	Lamy	0	4
27	1994	44	Olivier	Panis	9	16
31	1994	101	David	Brabham	0	16
3	1994	102	Ayrton	Senna	0	3
1	1994	84	Martin	Brundle	16	16
17	1994	56	Eddie	Irvine	6	13
22	1994	50	Jos	Verstappen	10	10

Figura 3: Puntuacions per pilot de cada temporada

7 Anàlisi de les dades

Gràcies a les queries hem pogut obtenir els CSV que contenen les dades necessàries per realitzar l'estudi que hem proposat. Aquestes dades són les puntuacions per any i per equips. I les puntuacions per any i per conductor. A partir dels dos CSV generats a l'hora de fer l'exportació de les taules generades per les queries 2 i 3, hem pogut, mitjançant el codi que trobem a la secció A.2, obtenir el **coeficient de variació** de les diferents temporades i eres.

Ara volem calcular una puntuació ajustada. És a dir, una puntuació que incorpora una penalització en aquells equips on predomina l'habilitat del conductor per damunt del treball de l'equip d'enginyeria.

Per determinar si el conductor és més predominant que els altres, definim la *penalty* d'un equip.

$$\text{penalty}_{equip} = \frac{\mu_{equip}}{\mu_{equip} + \sigma_{equip}}$$

A continuació s'explica el procediment per obtenir els valors σ_{equip} i μ_{equip}.

En primer lloc, cal entendre que a la Formula 1 els equips poden tenir més de 2 pilots participant en una temporada. Ara bé, cada carrera només la poden disputar 2 pilots per equip. Evidentment, la gran majoria d'equips es mantenen amb 2 pilots durant tota la temporada, però s'han de tractar aquests casos extrems de forma que el *penalty* sigui el més just possible pels equips. Per tant, tant la mitjana com la desviació estàndard es calcularan a partir dels punts per carrera de cada pilot de l'equip. Es defineix:

$$\text{Punts/carrera pilot} = \frac{\text{Puntuació total del pilot a l'any}}{\text{Nombre de curses realitzades a l'any}}$$

Gràcies a aquests punts per carrera de cada pilot, la μ_{equip} serà més baixa, per tant la σ_{equip} baixarà, provocant que el *penalty* sigui més just.

Conceptualment, definim μ_{equip} com la mitjana de puntuació per carrera dels pilots de l'equip en particular. Amb això es busca una forma d'expressar quina puntuació hauria de tenir un pilot estàndard del equip. Notem que a una mateixa carrera només poden còrrer 2 pilots per cada equip. Per tant, per poder calcular aquesta mitjana, el que es necessita és considerar el nombre de gran premis que hi ha a la temporada i multiplicar-lo per 2, d'aquesta forma s'obtenen el nombre de carreres que han corregut els cotxes dels equips. Per tant, definim:

$$\mu_{equip} = \frac{\sum_{i=1}^{m} n_i p_i}{n}$$

on m és el nombre de pilots que han corregut per l'equip a la temporada, n_i el nombre de carreres que ha fet el pilot a la temporada, p_i els punts per carrera del pilot a la temporada i $n = 2k$ el nombre de carreres realitzades pels cotxes de la temporada, on k és el nombre de grans premis realitzats a la temporada.

Una vegada es té μ_{equip}, es pot calcular σ_{equip}. En aquest cas, es vol veure quina és la desviació estàndard de les puntuacions dels pilots a les carreres. Per això, es farà ús de la puntuació mitjana del pilot i de la μ_{equip}. Definirem p_i, n_i i n com abans. Notem que les diferències

$$(p_i - \mu_{equip})^2$$

tindran sumands repetits com abans, ja que hi ha un sumand per cada carrera realitzada per un pilot. Per tant, definim:

$$\sigma_{equip} = \sqrt{\frac{\sum_{i=1}^{m}(p_i - \mu_{equip})^2}{n}}$$

Per tant, ja estem en condicions de calcular el *penalty*. Aleshores, una vegada es té el penalty_{equip}, s'aplica al nombre de punts que ha aconseguit l'equip a la temporada. Per tant, s'obté:

$$\text{PuntuacioAjustada}_{equip} = \text{PuntuacioTemporada}_{equip} \times \text{penalty}_{equip}$$

D'aquesta forma, es té la puntuació que l'equip realment mereix per la feina que ha exercit l'equip d'enginyeria.

Amb aquestes puntuacions es treballarà per contestar a la pregunta principal plantejada als objectius: Quina de les normatives va ser la que va provocar una competició més ajustada?

Per tant, primer es calcula la mitjana de les puntuacions ajustades dels equips a la temporada/any:

$$\mu_{any} = \frac{\sum_{i=1}^{l} P_i}{l}$$

on l és el nombre d'equips participants a la temporada i P_i les puntuacions ajustades dels equips a la temporada.

Seguidament, es calcula la desviació estàndard:

$$\sigma_{any} = \sqrt{\frac{\sum_{i=1}^{l}(P_i - \mu_{any})^2}{l}}$$

Arribats a aquest punt, cal recordar el que es va parlar respecte les puntuacions. Hom podria pensar que amb la desviació estàndard calculada ja es podria contestar a la pregunta, però les puntuacions donades als pilots en una carrera han anat variant al llarg del temps. Per tant, aquestes afecten a la desviació estàndard de les diferents temporades, fins i tot entre temporades de la mateixa normativa. Per tant, es considera fer ús del coeficient de variació de la temporada. Això es defineix com:

$$CV_{any} = \frac{\sigma_{any}}{\mu_{any}}$$

Per tant, ara tenim una dada (coeficient de variació) que no depèn dels diferents tipus de puntuació. En particular notem que l'estudi roman el mateix, ja que l'any/etapa que tingui el CV més baix serà el més competitiu, ja que la desviació estàndard és més baixa.

Ara tenint en compte els anys de cada normativa, per poder comparar les normatives (que tenen diferent durada d'anys) es calcularà la mitjana dels coeficients de variació de les temporades corresponents a l'etapa. Per tant:

$$\text{Score Etapa} = \frac{\sum_{i=1}^{n_k} CV_i}{n_k}$$

on n_k representa el nombre d'anys de l'etapa k, $k = 1, \ldots, 5$ i CV_i el coeficient de variació de l'any i de l'etapa k.

El següent gràfic és una representació dels CV dels diferents anys, on les línies discontínues són la mitjana

dels CV de l'etapa:

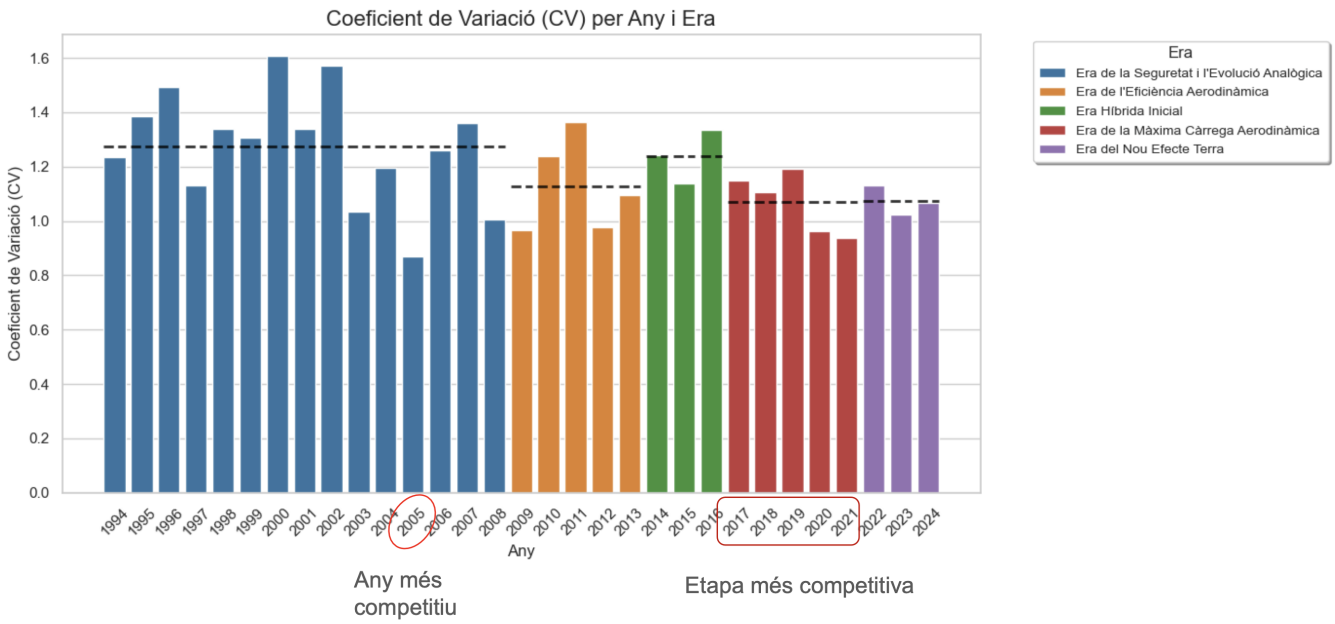


Figura 4: Coeficients de variació de les temporades del 1994 al 2024

Amb això, es podrien contestar diverses preguntes. La primera pregunta podria ser: Quin any ha estat el més competitiu en els equips d'enginyeria?

Doncs, segons marca el gràfic i tal com s'ha pogut comprovar amb el programa creat, l'any més competitiu va ser el **2005**, amb un $CV = 0.8697$

Per altra banda, es pot contestar a la pregunta principal del projecte: Quina de les normatives va ser la que va provocar una competició més ajustada?

El gràfic es correspon amb la taula següent:

Era de la Seguretat i l'Evolució Analògica (1994-2008)		
Any	Guanyador Real	Guanyador Merescut
1994	Williams	Williams
1995	Benetton	Benetton
1996	Williams	Williams
1997	Williams	Williams
1998	McLaren	McLaren
1999	Ferrari	McLaren
2000	Ferrari	McLaren
2001	Ferrari	Ferrari
2002	Ferrari	Ferrari
2003	Ferrari	Ferrari
2004	Ferrari	Ferrari
2005	Renault	McLaren
2006	Renault	Ferrari
2007	McLaren	McLaren
2008	Ferrari	Ferrari

Taula 2: Guanyadors reals i ajustats segons el model de penalització per a l'Era de la Seguretat i l'Evolució Analògica (1994-2008).

Era de l'Eficiència Aerodinàmica (2009-2013)		
Any	Guanyador Real	Guanyador Merescut
2009	Brawn	Brawn
2010	Red Bull	Red Bull
2011	Red Bull	Red Bull
2012	Red Bull	Red Bull
2013	Red Bull	Red Bull

Taula 3: Guanyadors reals i ajustats segons el model de penalització per a l'Era de l'Eficiència Aerodinàmica (2009-2013).

Era Híbrida Inicial (2014-2016)		
Any	Guanyador Real	Guanyador Merescut
2014	Mercedes	Mercedes
2015	Mercedes	Mercedes
2016	Mercedes	Mercedes

Taula 4: Guanyadors reals i ajustats segons el model de penalització per a l'Era Híbrida Inicial (2014-2016).

Per tant, obtenim la següent taula de resultats, on obtenim el percentatge de correlació entre el guanyador real i el guanyador merescut d'equips:

A partir de les taules i resultats que hem obtingut, es pot veure que la correlació des del 2009 fins l'any 2024 és del 100% , mentre que a l'Era de la Seguretat i l'Evolució Analògica (1994-2008) el percentatge és del 73%. Això es relaciona fortament amb una evidència clara en els últims anys de la F1: La tecnologia avança

Era de la Màxima Càrrega Aerodinàmica (2017-2021)		
Any	Guanyador Real	Guanyador Merescut
2017	Mercedes	Mercedes
2018	Mercedes	Mercedes
2019	Mercedes	Mercedes
2020	Mercedes	Mercedes
2021	Mercedes	Mercedes

Taula 5: Guanyadors reals i ajustats segons el model de penalització per a l'Era de la Màxima Càrrega Aerodinàmica (2017-2021).

Era del Nou Efecte Terra (des de 2022)		
Any	Guanyador Real	Guanyador Merescut
2022	Red Bull	Red Bull
2023	Red Bull	Red Bull
2024	McLaren	McLaren

Taula 6: Guanyadors reals i ajustats segons el model de penalització per a l'Era del Nou Efecte Terra (des de 2022).

Era	**Rate de coincidència**
Era de la Seguretat i l'Evolució Analògica (1994-2008)	11/15 = 0.73
Era de l'Eficiència Aerodinàmica (2009-2013)	5/5 = 1.00
Era Híbrida Inicial (2014-2016)	3/3 = 1.00
Era de la Màxima Càrrega Aerodinàmica (2017-2021)	5/5 = 1.00
Era del Nou Efecte Terra (des de 2022)	3/3 = 1.00

Taula 7: Rate de coincidència entre guanyador real i guanyador merescut per era

i tenir un cotxe molt ben desenvolupat juga un paper molt important per poder aconseguir la victòria, un paper sovint més important que el del pilot.

Per altra banda, veiem que no hi ha correlacions en 4 anys en particular, que són el 1999, 2000, 2005 i 2006. Com a curiositat, cal destacar que Fernando Alonso va guanyar els seus 2 mundials els anys 2005 i 2006 en l'equip Renault. Tal com es pot observar a la taula, aquests 2 anys Renault també va guanyar el campionat d'equips, però aplicant la nostra penalització, s'observa que no era el merescut guanyador en termes d'enginyeria. També cal destacar (com hem vist abans) que el 2005 va ser l'any més competitiu en termes d'enginyeria de les últimes 30 temporades. Per tant, hom pot pensar que en el cas de Fernando Alonso, tots dos mundials van ser més per mèrit del propi pilot que del propi cotxe, és a dir, el treball de l'equip d'enginyeria.

8 Conclusions i contribució dels membres

Aquest projecte ens ha permès donar resposta, mitjançant dades, a sospites i intuïcions que teníem sobre la Fórmula 1. Ha estat una experiència molt motivadora, ja que hem pogut treballar amb dades relacionades amb un tema que ens apassiona.

A nivell tècnic, el projecte ens ha ajudat a aprofundir en l'ús de consultes avançades de SQL, així com en la integració de la base de dades amb eines externes com Python. A més, el fet que les dades estiguessin en format CSV ens ha permès familiaritzar-nos amb aquest tipus d'arxius i entendre amb més profunditat com funcionen i com es poden processar.

Pel que fa a l'anàlisi, hem pogut identificar relacions molt interessants que no hauríem detectat a simple vista. En particular, hem pogut corroborar que la temporada 2005, on el pilot Fernando Alonso va guanyar el mundial, va fer-ho tot i tenir un cotxe que no destacava per damunt de la resta d'equips. Demostrant així, un gran nivell del conductor asturià.

Finalment, tot i que hem tingut dificultats a l'hora de definir una mesura de "score" ideal, a causa de les diferents escales que presentaven les dades, hem resolt el problema mantenint sempre la correcció matemàtica com a prioritat. Considerem que hem dut a terme una anàlisi rigorosa, utilitzant eines estadístiques per dotar de solidesa els càlculs i les conclusions obtingudes.

La contribució dels membres ha estat la següent:

- **Jan Accensi:** Anàlisi de dades i conclusions.
- **David Armillas:** Introducció, model E-R, part informe.
- **Alex Cervantes:** Importació de dades i consultes SQL.
- **Cristian Prado:** Consultes SQL, anàlisi de dades i redacció de l'anàlisi de Dades.

A Codi Python pel tractament de dades

A.1 Codi per passar CSV a Workbench

```
import pandas as pd
import mysql.connector
import os

schema_path = "f1db_tables_final.sql"

# Configuraci n de conexi n
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="****", # Poner la contrase a de tu db
)

# Crear un cursor para ejecutar consultas
cursor = conn.cursor()

# Leer el archivo SQL y ejecutar las consultas
with open(schema_path, "r", encoding="utf-8") as f:
    sql_script = f.read()

# Ejecutar m ltiples sentencias separadas por ";"
for statement in sql_script.split(";"):
    stmt = statement.strip()
    if stmt:
        try:
            cursor.execute(stmt)
        except mysql.connector.Error as err:
            print(f"Error ejecutando sentencia: {stmt}\n{err}")

conn.commit()
cursor.close()
conn.close()

# Conectar a la base
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="****",
    database="f1db"
)
cursor = conn.cursor()

# Esto sirve para limpiar la db, pero no es necesario
# era necesario para la prueba
```

```
        '''
        def truncate_all_tables():
            cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
            for table in reversed(list(tables.keys())):
                cursor.execute(f"TRUNCATE TABLE {table}")
            cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
            conn.commit()
            print("All tables truncated.")
        '''

        # Aqu  se definen las columnas a escoger de cada tabla
        tables = {
            "Constructors": ["constructorId", "name", "nationality"],
            "Drivers": ["driverId", "number", "code", "forename", "surname", "dob", ...
                "nationality"],
            "Circuits": ["circuitId", "name", "location", "country", "latitude", ...
                "longitude", "altitude"],
            "Races": ["raceId", "year", "round", "circuitId", "name", "date", "time"],
            "Results": ["resultId", "raceId", "driverId", "constructorId", "number", ...
                "grid", "position", "positionText", "points", "laps", "time", ...
                "milliseconds", "fastestLap", "fastestLapTime", "fastestLapSpeed"],
            "SprintResults": ["sprintResultId", "raceId", "driverId", "constructorId", ...
                "number", "grid", "position", "positionText", "points", "laps", "time", ...
                "milliseconds", "fastestLap", "fastestLapTime"],
            "LapTimes": ["raceId", "driverId", "lap", "position", "time", "milliseconds"],
            "ConstructorStandings": ["raceId", "constructorId", "points", "position", ...
                "positionText", "wins"],
            "DriverStandings": ["raceId", "driverId", "points", "position", ...
                "positionText", "wins"]
        }

        # Asocia nombre de archivo con tabla, para facilitar la lectura
        file_map = {
            "Drivers": "drivers.csv",
            "Constructors": "constructors.csv",
            "Circuits": "circuits.csv",
            "Races": "races.csv",
            "Results": "results.csv",
            "SprintResults": "sprint_results.csv",
            "LapTimes": "lap_times.csv",
            "ConstructorStandings": "constructor_standings.csv",
            "DriverStandings": "driver_standings.csv"
        }

        # Se puede quitar
        #truncate_all_tables()

        # Procesar cada tabla
        for table, valid_cols in tables.items():
```

```
            # Obtener el nombre del archivo correspondiente
            data_folder = "data"
            file_name = os.path.join(data_folder, file_map.get(table))

            # Comprobar si el archivo existe
            if not os.path.exists(file_name):
                print(f"Archivo no encontrado para {table}: {file_name}")

            print(f"Insertando en tabla {table} desde {file_name}...")

            # Leer el archivo CSV
            # Se a ade el argumento na_values para manejar los valores nulos
            df = pd.read_csv(file_name, na_values='\\N')

            # Manejo especial para SprintResults: a adir sprintResultId , ya que de por ...
                s   no existe
            if table == "SprintResults" and "sprintResultId" not in df.columns:
                print("A adiendo sprintResultId autogenerado...")
                df.insert(0, "sprintResultId", range(1, len(df) + 1))

            # Filtrar columnas v lidas que existen en el archivo
            df = df[[col for col in valid_cols if col in df.columns]]

            # Eliminar duplicados por clave primaria (primera columna del schema)
            df = df.drop_duplicates(subset=df.columns[0], keep="first")

            # Reemplazar NaN con None (importante para MySQL)
            df = df.where(pd.notnull(df), None)

            # Construcci n din mica de columnas y placeholders
            insert_cols = df.columns.tolist()
            col_names = ', '.join([f"`{col}`" for col in insert_cols])
            placeholders = ', '.join(['%s'] * len(insert_cols))

            # Por cada fila
            for _, row in df.iterrows():

                # Crear una tupla con los valores de la fila
                row_tuple = tuple([None if pd.isna(val) else val for val in row])

                # Insertar en la tabla
                cursor.execute(
                    f"INSERT INTO {table} ({col_names}) VALUES ({placeholders})",
                    row_tuple
                )
            # Commit de la transacci n
            conn.commit()
            print(f"Insertados {len(df)} registros en {table}")

```

```
        # Cerrar la conexi n
        cursor.close()
        # Cerrar la conexi n
        conn.close()
        print("Todos los datos insertados correctamente.")
```

A.2 Codis per l'anàlisi

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Cargar los datos
print("Cargando datos...")
pilots_df = pd.read_csv("data/Puntuacio_Pilots_BD.csv")
equips_df = pd.read_csv("data/Puntuacio_Equips_BD.csv")

# Definir funci n para asignar era por a o
def assignar_era(year):
    if 1994 <= year <= 2008:
        return "Era de la Seguretat i l'Evoluci  Anal gica"
    elif 2009 <= year <= 2013:
        return "Era de l'Efici ncia Aerodin mica"
    elif 2014 <= year <= 2016:
        return "Era H brida Inicial"
    elif 2017 <= year <= 2021:
        return "Era de la M xima C rrega Aerodin mica"
    elif year >= 2022:
        return "Era del Nou Efecte Terra"
    else:
        return "Fora de per ode"

pilots_df["era"] = pilots_df["year"].apply(assignar_era)
equips_df["era"] = equips_df["year"].apply(assignar_era)

# Calcular puntos por carrera para cada piloto
pilots_df["puntos_por_carrera"] = pilots_df["total_points"] / pilots_df["num_races"]

# Calcular estad sticas para cada equipo por a o
print("Calculando estad sticas por equipo...")
equips_stats = []

for year in sorted(pilots_df["year"].unique()):
    equipos_a o = {}

    # Obtener equipos de este a o
    teams_year = equips_df[equips_df["year"] == year]
```

```

        for constructor_id in teams_year["constructorId"]:
            # Obtener pilotos y puntos de este equipo y a o
            pilotos_equipo = pilots_df[(pilots_df["year"] == year) & ...
                (pilots_df["constructorId"] == constructor_id)]

            # Puntos totales del equipo seg n equipos_df
            equipo_info = teams_year[teams_year["constructorId"] == constructor_id].iloc[0]
            puntos_equipo = equipo_info["total_points"]
            equipo_nombre = equipo_info["constructor_name"]

            # Si no hay pilotos registrados, continuar
            if len(pilotos_equipo) == 0:
                continue

            # Calcular el n mero total de carreras disputadas por el equipo
            total_carreras_pilotos = pilotos_equipo["num_races"].sum()
            # Dividir entre 2 ya que en cada GP participan 2 coches por equipo
            total_carreras_equipo = total_carreras_pilotos / 2

            # Calcular media ponderada de puntos por carrera
            # Cada piloto contribuye proporcionalmente al n mero de carreras disputadas
            suma_ponderada = sum(pilotos_equipo["puntos_por_carrera"] * ...
                pilotos_equipo["num_races"])
            media_puntos_por_carrera = suma_ponderada / total_carreras_pilotos

            # Calcular desviaci n est ndar ponderada
            # Para cada piloto, calcular (pts_por_carrera - media)^2 * num_carreras
            suma_cuadrados = sum(
                (pilotos_equipo["puntos_por_carrera"] - media_puntos_por_carrera)**2 * ...
                    pilotos_equipo["num_races"]
            )
            # Dividir entre el total de carreras para obtener la varianza ponderada
            varianza = suma_cuadrados / total_carreras_pilotos
            std_puntos_por_carrera = np.sqrt(varianza)

            # Calcular factor de penalizaci n usando puntos por carrera
            if media_puntos_por_carrera > 0:
                factor_penalizacion = media_puntos_por_carrera / (std_puntos_por_carrera + ...
                    media_puntos_por_carrera)
            else:
                factor_penalizacion = 1  # Sin penalizaci n si no hay puntos

            # Calcular puntos ajustados
            puntos_ajustados = puntos_equipo * factor_penalizacion

            # Guardar estad sticas del equipo
            equipos_a o[constructor_id] = {
                "nombre": equipo_nombre,
```

```
                "puntos_totales": puntos_equipo,
                "std_puntos_por_carrera": std_puntos_por_carrera,
                "media_puntos_por_carrera": media_puntos_por_carrera,
                "factor_penalizacion": factor_penalizacion,
                "puntos_ajustados": puntos_ajustados,
                "num_pilotos": len(pilotos_equipo),
                "total_carreras_pilotos": total_carreras_pilotos,
                "total_carreras_equipo": total_carreras_equipo
            }

            # Informaci n de diagn stico para 2024
            if year == 2006:
                print(f"\nEquipo: {equipo_nombre}")
                print("  Pilotos (puntos y carreras):")
                for _, p in pilotos_equipo.iterrows():
                    print(f"    {p['forename']} {p['surname']}: {p['total_points']} pts en ...
                        {p['num_races']} carreras = {p['puntos_por_carrera']:.2f} pts/carrera")
                print(f"  Total carreras con pilotos: {total_carreras_pilotos}, Total ...
                    carreras equipo: {total_carreras_equipo:.1f}")
                print(f"  Media pts/carrera ponderada: {media_puntos_por_carrera:.2f}, ...
                    Desv.Std ponderada: {std_puntos_por_carrera:.2f}")
                print(f"  Factor penalizaci n: {factor_penalizacion:.4f}")
                print(f"  Puntos originales: {puntos_equipo}, Puntos ajustados: ...
                    {puntos_ajustados:.1f}")

        # Calcular estad sticas de la temporada
        if equipos_a o:
            puntos_ajustados_valores = [stats["puntos_ajustados"] for stats in ...
                equipos_a o.values()]
            media_temporada = np.mean(puntos_ajustados_valores)
            std_temporada = np.std(puntos_ajustados_valores, ddof=0)

            # Calcular coeficiente de variaci n
            cv_temporada = std_temporada / media_temporada if media_temporada > 0 else 0

            equips_stats.append({
                "year": year,
                "era": assignar_era(year),
                "num_equipos": len(equipos_a o),
                "media_puntos_ajustados": media_temporada,
                "std_puntos_ajustados": std_temporada,
                "cv": cv_temporada
            })

    # Convertir a DataFrame
    stats_df = pd.DataFrame(equips_stats)

    # Crear DataFrame amb punts ajustats per equip i any
    print("Creant DataFrame amb punts ajustats per equip...")
```

```
    punts_ajustats_data = []

    for year in sorted(pilots_df["year"].unique()):
        teams_year = equips_df[equips_df["year"] == year]

        for constructor_id in teams_year["constructorId"]:
            pilotos_equipo = pilots_df[(pilots_df["year"] == year) & ...
                (pilots_df["constructorId"] == constructor_id)]

            if len(pilotos_equipo) == 0:
                continue

            equipo_info = teams_year[teams_year["constructorId"] == constructor_id].iloc[0]
            puntos_equipo = equipo_info["total_points"]
            equipo_nombre = equipo_info["constructor_name"]

            # Recalcular punts ajustats (mateix c lcul que abans)
            total_carreras_pilotos = pilotos_equipo["num_races"].sum()
            suma_ponderada = sum(pilotos_equipo["puntos_por_carrera"] * ...
                pilotos_equipo["num_races"])
            media_puntos_por_carrera = suma_ponderada / total_carreras_pilotos

            suma_cuadrados = sum(
                (pilotos_equipo["puntos_por_carrera"] - media_puntos_por_carrera)**2 * ...
                    pilotos_equipo["num_races"]
            )
            varianza = suma_cuadrados / total_carreras_pilotos
            std_puntos_por_carrera = np.sqrt(varianza)

            if media_puntos_por_carrera > 0:
                factor_penalizacion = media_puntos_por_carrera / (std_puntos_por_carrera + ...
                    media_puntos_por_carrera)
            else:
                factor_penalizacion = 1

            puntos_ajustados = puntos_equipo * factor_penalizacion

            punts_ajustats_data.append({
                "year": year,
                "era": assignar_era(year),
                "constructorId": constructor_id,
                "constructor_name": equipo_nombre,
                "puntos_originales": puntos_equipo,
                "puntos_ajustados": puntos_ajustados,
                "factor_penalizacion": factor_penalizacion
            })

    # Convertir a DataFrame i guardar
    punts_ajustats_df = pd.DataFrame(punts_ajustats_data)
```

```
    punts_ajustats_df.to_csv("puntuacio_equips_penalitzada.csv", index=False)

    print(f"Guardat 'puntuacio_equips_penalitzada.csv' amb {len(punts_ajustats_df)} ...
        registres")

    # Calcular CV por era (media de los CV de cada a o en la misma era)
    print("\nCalculando CV por era...")
    cv_por_era = stats_df.groupby("era")["cv"].mean().reset_index()

    # Guardar resultados
    stats_df.to_csv("cv_per_any_ponderado.csv", index=False)
    cv_por_era.to_csv("cv_per_era_ponderado.csv", index=False)

    print("\nResultados del CV por era:")
    print(cv_por_era)

    # Visualizaci n
    plt.figure(figsize=(12, 6))
    plt.bar(cv_por_era["era"], cv_por_era["cv"])
    plt.title("Coeficiente de Variaci n Medio por Era de F1 (Ponderado por Carreras)")
    plt.ylabel("CV (Coeficiente de Variaci n)")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.savefig("cv_per_era_ponderado.png")
    print("Gr fico guardado como 'cv_per_era_ponderado.png'")
```

B Codi per crear la base de dades a Workbench

```
        SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
        SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
        SET @OLD_SQL_MODE=@@SQL_MODE, ...
            SQL_MODE='STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
        ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

        DROP SCHEMA IF EXISTS `f1db`;
        CREATE SCHEMA IF NOT EXISTS `f1db` DEFAULT CHARACTER SET utf8mb4 COLLATE ...
            utf8mb4_unicode_ci;
        USE f1db;

        -- CIRCUITS
        CREATE TABLE IF NOT EXISTS `f1db`.`circuits` (
          `circuitId` INT NOT NULL,
          `name` VARCHAR(255) NOT NULL,
          `location` VARCHAR(255),
          `country` VARCHAR(255),
          `latitude` FLOAT,
```

```sql
  `longitude` FLOAT,
  `altitude` INT,
  CONSTRAINT `circuit_pkey` PRIMARY KEY (`circuitId`)
);

-- CONSTRUCTORS
CREATE TABLE IF NOT EXISTS `f1db`.`constructors` (
  `constructorId` INT NOT NULL,
  `name` VARCHAR(255) NOT NULL,
  `nationality` VARCHAR(255),
  CONSTRAINT `constructor_pkey` PRIMARY KEY (`constructorId`)
);

-- DRIVERS
CREATE TABLE IF NOT EXISTS `f1db`.`drivers` (
  `driverId` INT NOT NULL,
  `number` INT,
  `code` VARCHAR(3),
  `forename` VARCHAR(255) NOT NULL,
  `surname` VARCHAR(255) NOT NULL,
  `dob` DATE,
  `nationality` VARCHAR(255),
  CONSTRAINT `driver_pkey` PRIMARY KEY (`driverId`)
);

-- RACES
CREATE TABLE IF NOT EXISTS `f1db`.`races` (
  `raceId` INT NOT NULL,
  `year` INT NOT NULL,
  `round` INT NOT NULL,
  `circuitId` INT NOT NULL,
  `name` VARCHAR(255) NOT NULL,
  `date` DATE,
  `time` TIME,
  CONSTRAINT `race_pkey` PRIMARY KEY (`raceId`)
);

-- DRIVER STANDINGS
CREATE TABLE IF NOT EXISTS `f1db`.`driverStandings` (
  `raceId` INT NOT NULL,
  `driverId` INT NOT NULL,
  `points` FLOAT NOT NULL DEFAULT 0,
  `position` INT,
  `positionText` VARCHAR(255),
  `wins` INT DEFAULT 0
);

-- CONSTRUCTOR STANDINGS
CREATE TABLE IF NOT EXISTS `f1db`.`constructorStandings` (
```

```sql
          `raceId` INT NOT NULL,
          `constructorId` INT NOT NULL,
          `points` FLOAT NOT NULL DEFAULT 0,
          `position` INT,
          `positionText` VARCHAR(255),
          `wins` INT DEFAULT 0
        );

        -- RESULTS
        CREATE TABLE IF NOT EXISTS `f1db`.`results` (
          `resultId` INT NOT NULL,
          `raceId` INT NOT NULL,
          `driverId` INT NOT NULL,
          `constructorId` INT NOT NULL,
          `number` INT,
          `grid` INT,
          `position` INT,
          `positionText` VARCHAR(255),
          `points` FLOAT,
          `laps` INT,
          `time` VARCHAR(255),
          `milliseconds` INT,
          `fastestLap` INT,
          `rank` INT,
          `fastestLapTime` VARCHAR(255),
          `fastestLapSpeed` VARCHAR(255),
          CONSTRAINT `results_pkey` PRIMARY KEY (`resultId`)
        );

        -- SPRINT RESULTS
        CREATE TABLE IF NOT EXISTS `f1db`.`sprintResults` (
          `sprintResultId` INT NOT NULL,
          `raceId` INT NOT NULL,
          `driverId` INT NOT NULL,
          `constructorId` INT NOT NULL,
          `number` INT,
          `grid` INT,
          `position` INT,
          `positionText` VARCHAR(255),
          `points` FLOAT,
          `laps` INT,
          `time` VARCHAR(255),
          `milliseconds` INT,
          `fastestLap` INT,
          `fastestLapTime` VARCHAR(255),
          CONSTRAINT `sprintresults_pkey` PRIMARY KEY (`sprintResultId`)
        );

        -- LAP TIMES
```

```sql
CREATE TABLE IF NOT EXISTS `f1db`.`lapTimes` (
  `raceId` INT NOT NULL,
  `driverId` INT NOT NULL,
  `lap` INT NOT NULL,
  `position` INT,
  `time` VARCHAR(255),
  `milliseconds` INT,
  CONSTRAINT `laptimes_pkey` PRIMARY KEY (`raceId`, `driverId`, `lap`)
);

-- CLAVES FORNEAS
ALTER TABLE `f1db`.`races`
  ADD CONSTRAINT `races_fk_circuit` FOREIGN KEY (`circuitId`) REFERENCES ...
      `f1db`.`circuits` (`circuitId`) ON DELETE CASCADE;

ALTER TABLE `f1db`.`driverStandings`
  ADD CONSTRAINT `ds_fk_race` FOREIGN KEY (`raceId`) REFERENCES `f1db`.`races` ...
      (`raceId`) ON DELETE CASCADE,
  ADD CONSTRAINT `ds_fk_driver` FOREIGN KEY (`driverId`) REFERENCES ...
      `f1db`.`drivers` (`driverId`) ON DELETE CASCADE;

ALTER TABLE `f1db`.`constructorStandings`
  ADD CONSTRAINT `cs_fk_race` FOREIGN KEY (`raceId`) REFERENCES `f1db`.`races` ...
      (`raceId`) ON DELETE CASCADE,
  ADD CONSTRAINT `cs_fk_constructor` FOREIGN KEY (`constructorId`) REFERENCES ...
      `f1db`.`constructors` (`constructorId`) ON DELETE CASCADE;

ALTER TABLE `f1db`.`results`
  ADD CONSTRAINT `results_fk_race` FOREIGN KEY (`raceId`) REFERENCES ...
      `f1db`.`races` (`raceId`) ON DELETE CASCADE,
  ADD CONSTRAINT `results_fk_driver` FOREIGN KEY (`driverId`) REFERENCES ...
      `f1db`.`drivers` (`driverId`) ON DELETE CASCADE,
  ADD CONSTRAINT `results_fk_constructor` FOREIGN KEY (`constructorId`) REFERENCES ...
      `f1db`.`constructors` (`constructorId`) ON DELETE CASCADE;

ALTER TABLE `f1db`.`sprintResults`
  ADD CONSTRAINT `sprint_fk_race` FOREIGN KEY (`raceId`) REFERENCES `f1db`.`races` ...
      (`raceId`) ON DELETE CASCADE,
  ADD CONSTRAINT `sprint_fk_driver` FOREIGN KEY (`driverId`) REFERENCES ...
      `f1db`.`drivers` (`driverId`) ON DELETE CASCADE,
  ADD CONSTRAINT `sprint_fk_constructor` FOREIGN KEY (`constructorId`) REFERENCES ...
      `f1db`.`constructors` (`constructorId`) ON DELETE CASCADE;

ALTER TABLE `f1db`.`lapTimes`
  ADD CONSTRAINT `laptime_fk_race` FOREIGN KEY (`raceId`) REFERENCES ...
      `f1db`.`races` (`raceId`) ON DELETE CASCADE,
  ADD CONSTRAINT `laptime_fk_driver` FOREIGN KEY (`driverId`) REFERENCES ...
      `f1db`.`drivers` (`driverId`) ON DELETE CASCADE;
```

```

        SET SQL_MODE=@OLD_SQL_MODE;
        SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
        SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
```